

RAG IMPLEMENTATION GUIDE

Complete Code Examples & Best Practices

Created: May 27, 2026

TABLE OF CONTENTS

1. Project Setup & Installation
2. Basic RAG with LangChain
3. RAG with LlamaIndex
4. Advanced Chunking Strategies
5. Implementing Reranking
6. Hybrid Retrieval (BM25 + Semantic)
7. Production RAG API
8. Monitoring & Evaluation
9. Cost Optimization
10. Common Patterns & Templates

1. PROJECT SETUP & INSTALLATION

1.1 Virtual Environment Setup

```
python -m venv rag_env  
source rag_env/bin/activate # On Windows: rag_env\Scripts\activate
```

Upgrade pip:

```
pip install --upgrade pip
```

1.2 Core Dependencies

Essential packages:

```
pip install langchain openai python-dotenv  
pip install sentence-transformers chromadb  
pip install pypdf python-docx markdown  
pip install numpy pandas scikit-learn
```

For production:

```
pip install fastapi uvicorn  
pip install redis  
pip install python-multipart
```

1.3 Environment Configuration

Create .env file:

```
OPENAI_API_KEY=sk-...  
ANTHROPIC_API_KEY=sk-ant-...  
COHERE_API_KEY=...  
PINECONE_API_KEY=...  
PINECONE_ENVIRONMENT=us-west1
```

Load in code:

```
from dotenv import load_dotenv  
import os
```

```
load_dotenv()  
api_key = os.getenv("OPENAI_API_KEY")
```

2. BASIC RAG WITH LANGCHAIN

2.1 Simple Q&A; System

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chains import RetrievalQA
import os

# Load document
loader = PyPDFLoader("document.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", " ", "", ""])
chunks = splitter.split_documents(documents)

# Create embeddings
embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

# Store in ChromaDB
vectorstore = Chroma.from_documents(
    chunks,
    embeddings,
    persist_directory="./chroma_db"
)

# Create retriever
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3} # Retrieve top 3
)

# Create LLM chain
llm = OpenAI(temperature=0)
qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever
)

# Query
result = qa("What is machine learning?")
print(result)
```

2.2 RAG with Custom Prompt

```
from langchain.prompts import PromptTemplate
from langchain.chains import RetrievalQA

template = ""
You are a helpful assistant. Answer based ONLY on the context.
If not in context, say you don't know.

Context: {context}

Question: {question}

Answer:""

prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)

qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    chain_type_kwargs={"prompt": prompt}
)

result = qa("Your question here")
print(result['result'])
print(result['source_documents']) # Show sources
```

3. RAG WITH LLAMAINDEX

3.1 LlamaIndex Basic Setup

```
from llama_index import VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings import OpenAIEmbedding
from llama_index.llm import OpenAI

# Load documents
documents = SimpleDirectoryReader("./data").load_data()

# Create index
index = VectorStoreIndex.from_documents(
    documents,
    embed_model=OpenAIEmbedding(model="text-embedding-3-small")
)

# Create query engine
query_engine = index.as_query_engine()

# Query
response = query_engine.query("Your question")
print(response)

# Get sources
for node in response.source_nodes:
    print(node.text)
```

3.2 Advanced LlamaIndex with Reranking

```
from llama_index.postprocessor import SentenceTransformerRerank
from llama_index.retrievers import VectorIndexRetriever
from llama_index.query_engine import RetrieverQueryEngine

# Create reranker
reranker = SentenceTransformerRerank(
    model="cross-encoder/ms-marco-MiniLM-L-12-v2",
    top_n=2
)

# Create retriever
retriever = VectorIndexRetriever(index, similarity_top_k=10)

# Combine with reranker
query_engine = RetrieverQueryEngine(
    retriever=retriever,
    node_postprocessors=[reranker]
)

response = query_engine.query("Question")
print(response)
```

4. ADVANCED CHUNKING STRATEGIES

4.1 Semantic Chunking

```
from semantic_text_splitter import TextSplitter

# Uses embeddings to find semantic boundaries
splitter = TextSplitter()

chunks = splitter.chunks(text, chunk_size=1000)

# Advantages:
# - Preserves sentence/paragraph boundaries
# - No artificial split mid-topic
# - Better context preservation
#
# Disadvantages:
# - Slower (uses embeddings)
# - Chunk sizes less uniform
```

4.2 Document-Aware Chunking

```
def chunk_by_structure(document):
    chunks = []
    current_chunk = ""
    current_level = None

    for line in document.split('\n'):
        # Check if header
        if line.startswith('#'):
            if current_chunk:
                chunks.append(current_chunk)
            current_chunk = line
            current_level = len(line) - len(line.lstrip('#'))
        else:
            current_chunk += "\n" + line

    # Split if chunk too large
    if len(current_chunk) > 1000:
        chunks.append(current_chunk)
        current_chunk = ""

    if current_chunk:
        chunks.append(current_chunk)
    return chunks

# Preserves document structure and headers
```

4.3 Overlapping Chunks

```
# Chunk with overlap helps retrieval
chunk_size = 1000
overlap = 200
```

```
chunks = []  
for i in range(0, len(text), chunk_size - overlap):  
    chunk = text[i:i + chunk_size]  
    chunks.append(chunk)
```

```
# Overlap benefits:  
# - Prevents losing context at boundaries  
# - Increases retrieval coverage  
# - Small storage overhead
```

```
# Recommended: 10-20% overlap  
overlap_pct = 0.2
```

5. IMPLEMENTING RERANKING

5.1 Cross-Encoder Reranking

```
from sentence_transformers import CrossEncoder

# Load cross-encoder model
reranker = CrossEncoder(
    'cross-encoder/ms-marco-MiniLM-L-12-v2'
)

# Get initial retrieval results
retrieved_docs = retriever.get_relevant_documents(query)

# Create pairs for reranking
pairs = [[query, doc.page_content] for doc in retrieved_docs]

# Score all pairs
scores = reranker.predict(pairs)

# Rank by score
scored_docs = list(zip(retrieved_docs, scores))
scored_docs.sort(key=lambda x: x[1], reverse=True)

# Keep top K
final_docs = [doc for doc, score in scored_docs[:3]]

# Use final_docs for generation
```

5.2 LLM-Based Reranking

```
from langchain.llms import OpenAI

def llm_rerank(query, docs, llm):
    """Use LLM to rank retrieved docs"""
    ranking = []

    for i, doc in enumerate(docs):
        prompt = f"""
        Is this document relevant to the query?
        Query: {query}
        Document: {doc.page_content[:500]}
        Answer with just the score 1-10."""

        score = llm.predict(prompt)
        ranking.append((doc, float(score)))

    ranking.sort(key=lambda x: x[1], reverse=True)
    return [doc for doc, score in ranking]

# Usage
reranked = llm_rerank(query, retrieved_docs, llm)

# Note: Slower but very accurate
```

6. HYBRID RETRIEVAL (BM25 + SEMANTIC)

6.1 Combining Dense & Sparse Retrieval

```
from rank_bm25 import BM25Okapi
import numpy as np

def hybrid_search(query, documents, embeddings, k=5):
    """Combine BM25 and semantic similarity"""

    # BM25 retrieval
    corpus = [doc.split() for doc in documents]
    bm25 = BM25Okapi(corpus)
    bm25_scores = bm25.get_scores(query.split())

    # Semantic retrieval
    query_embedding = embed_model.encode(query)
    semantic_scores = []

    for doc_embed in embeddings:
        sim = cosine_similarity([query_embedding], [doc_embed])[0][0]
        semantic_scores.append(sim)

    # Normalize scores [0, 1]
    bm25_norm = bm25_scores / (np.max(bm25_scores) + 1e-10)
    semantic_norm = np.array(semantic_scores)

    # Combine (equal weights)
    combined = 0.5 * bm25_norm + 0.5 * semantic_norm

    # Get top K
    top_indices = np.argsort(combined)[-k:]::-1]
    return [documents[i] for i in top_indices]

# Usage
results = hybrid_search(query, docs, embeddings)
```

6.2 Reciprocal Rank Fusion

```
from collections import defaultdict

def reciprocal_rank_fusion(results_list, k=60):
    """Combine rankings from multiple systems"""

    scores = defaultdict(float)

    for results in results_list:
        for rank, doc_id in enumerate(results):
            scores[doc_id] += 1 / (k + rank + 1)

    sorted_docs = sorted(
        scores.items(),
        key=lambda x: x[1],
```

```
reverse=True  
)
```

```
return [doc_id for doc_id, score in sorted_docs]
```

```
# Usage
```

```
bm25_results = bm25_search(query)
```

```
semantic_results = semantic_search(query)
```

```
final_results = reciprocal_rank_fusion([bm25_results, semantic_results])
```

7. PRODUCTION RAG API

7.1 FastAPI RAG Service

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain.chains import RetrievalQA
import logging

app = FastAPI(title="RAG API")
logger = logging.getLogger(__name__)

# Load models once on startup
qa_chain = None

@app.on_event("startup")
async def startup_event():
    global qa_chain
    qa_chain = load_rag_chain() # Your RAG setup
    logger.info("RAG chain loaded")

class QueryRequest(BaseModel):
    query: str
    top_k: int = 3

class QueryResponse(BaseModel):
    answer: str
    sources: list

@app.post("/query", response_model=QueryResponse)
async def query(request: QueryRequest):
    try:
        result = qa_chain(request.query)
        return QueryResponse(
            answer=result['result'],
            sources=[
                doc.metadata.get('source', 'Unknown')
                for doc in result.get('source_documents', [])
            ]
        )
    except Exception as e:
        logger.error(f"Query failed: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

7.2 Adding Caching & Rate Limiting

```
from functools import lru_cache
from slowapi import Limiter
from slowapi.util import get_remote_address
import hashlib

limiter = Limiter(key_func=get_remote_address)
query_cache = {}

def cache_key(query: str) -> str:
    return hashlib.md5(query.encode()).hexdigest()

@app.post("/query")
@limiter.limit("10/minute")
async def query(request: QueryRequest):
    key = cache_key(request.query)

    # Check cache
    if key in query_cache:
        logger.info("Cache hit")
        return query_cache[key]

    # Generate answer
    result = qa_chain(request.query)
    response = QueryResponse(
        answer=result['result'],
        sources=[d.metadata.get('source')
                 for d in result.get('source_documents', [])]
    )

    # Cache result
    query_cache[key] = response
    return response
```

8. MONITORING & EVALUATION

8.1 Evaluation Framework

```
from sklearn.metrics import precision_recall_fscore_support
import json
```

```
class RAGEvaluator:
    def __init__(self, qa_chain):
        self.qa_chain = qa_chain
        self.results = []

    def evaluate(self, test_set):
        for query, expected_answer in test_set:
            result = self.qa_chain(query)
            answer = result['result']
            sources = result['source_documents']

            self.results.append({
                'query': query,
                'expected': expected_answer,
                'actual': answer,
                'sources': len(sources),
                'match': expected_answer in answer
            })

    def accuracy(self):
        correct = sum(1 for r in self.results if r['match'])
        return correct / len(self.results)

    def report(self):
        matches = sum(1 for r in self.results if r['match'])
        total = len(self.results)
        accuracy = matches / total

        print(f"Total: {total}")
        print(f"Correct: {matches}")
        print(f"Accuracy: {accuracy:.2%}")

    return self.results
```

8.2 Logging & Monitoring

```
import logging
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s'
)

logger = logging.getLogger(__name__)

class QueryLogger:
    def __init__(self, log_file="queries.jsonl"):
```

```

self.log_file = log_file

def log_query(self, query, response, latency, cost):
    log_entry = {
        'timestamp': datetime.now().isoformat(),
        'query': query,
        'response_length': len(response),
        'latency_ms': latency,
        'cost_usd': cost
    }

    with open(self.log_file, 'a') as f:
        f.write(json.dumps(log_entry) + '\n')

    logger.info(f"Query: {query[:50]}... Latency: {latency}ms")

# Usage
logger = QueryLogger()
start = time.time()
result = qa_chain(query)
latency = (time.time() - start) * 1000
cost = calculate_cost(result)
logger.log_query(query, result['result'], latency, cost)

```

9. COST OPTIMIZATION

9.1 Token Counting & Optimization

```
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-3.5-turbo")

def count_tokens(text):
    return len(encoding.encode(text))

# Optimize retrieval
def optimize_context(docs, max_tokens=3000):
    context = ""
    token_count = 0

    for doc in docs:
        doc_tokens = count_tokens(doc)
        if token_count + doc_tokens < max_tokens:
            context += doc + "\n"
            token_count += doc_tokens
        else:
            break

    return context

# Cost calculation
def estimate_cost(prompt_tokens, completion_tokens):
    prompt_cost = prompt_tokens * 0.0005 / 1000
    completion_cost = completion_tokens * 0.0015 / 1000
    return prompt_cost + completion_cost
```

9.2 Using Smaller Models

```
# Option 1: Use faster embedding models
from sentence_transformers import SentenceTransformer

# Fast (384 dims)
model = SentenceTransformer('all-MiniLM-L6-v2')

# Better quality but slower (768 dims)
# model = SentenceTransformer('all-mpnet-base-v2')

# Option 2: Use smaller LLMs
# Instead of GPT-4: Use GPT-3.5
# Instead of Claude 3: Use Claude Haiku
# Instead of paid APIs: Use open-source Llama 2

# Option 3: Batch processing
def batch_query(queries, batch_size=5):
    results = []
    for i in range(0, len(queries), batch_size):
        batch = queries[i:i+batch_size]
        batch_results = [qa_chain(q) for q in batch]
        results.extend(batch_results)
```

return results

10. COMMON PATTERNS & TEMPLATES

10.1 Template: PDF Q&A; System

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

def create_pdf_qa(pdf_path: str):
    # Load & split
    loader = PyPDFLoader(pdf_path)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)

    # Embed & store
    embeddings = HuggingFaceEmbeddings()
    vector_store = Chroma.from_documents(chunks, embeddings)

    # Create QA
    llm = OpenAI(temperature=0)
    qa = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type="stuff",
        retriever=vector_store.as_retriever(search_kwargs={"k": 3})
    )

    return qa

# Usage
qa = create_pdf_qa("document.pdf")
answer = qa("Your question")
```

10.2 Template: Web Search RAG

```
from langchain.tools import DuckDuckGoSearchRun
from langchain.agents import initialize_agent, Tool
from langchain.agents import AgentType
from langchain.llms import OpenAI

def create_web_rag():
    search = DuckDuckGoSearchRun()

    tools = [
        Tool(
            name="Web Search",
            func=search.run,
            description="Search the web"
        )
    ]
```

```

llm = OpenAI(temperature=0)
agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

```

```

return agent

```

```

# Usage
agent = create_web_rag()
result = agent.run("Latest news on AI?")
print(result)

```

10.3 Template: Conversational RAG

```

from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain

```

```

def create_conversational_rag(vectorstore):
    memory = ConversationBufferMemory(
        memory_key="chat_history",
        return_messages=True
    )

```

```

    llm = OpenAI(temperature=0)
    chain = ConversationalRetrievalChain.from_llm(
        llm=llm,
        retriever=vectorstore.as_retriever(),
        memory=memory
    )

```

```

return chain

```

```

# Usage
chain = create_conversational_rag(vector_store)

```

```

# Multi-turn conversation
result1 = chain({"question": "What is AI?"})
result2 = chain({"question": "Tell me more about that"}) # Remembers context
print(result2['answer'])

```

QUICK START CHECKLIST

Before Building RAG:

- Define your knowledge base (documents, size, format)
- Choose LLM (OpenAI, Claude, open-source)
- Select vector DB (Pinecone, Chroma, Milvus)
- Pick embedding model (MiniLM, MPNet, OpenAI)
- Plan chunking strategy (size, overlap)
- Create test dataset (10-20 Q&A; pairs)

During Development:

- Start with basic implementation
- Evaluate retrieval quality (recall, precision)
- Tune chunking parameters
- Add reranking if quality insufficient
- Optimize prompts
- Test edge cases

Before Production:

- Evaluate end-to-end system
- Optimize for latency and cost
- Add monitoring and logging
- Implement caching
- Security and access control
- Load testing

Common Issues & Solutions:

Poor Retrieval → Better chunks + hybrid search + reranking
High Latency → Caching + smaller models + async
High Cost → Fewer docs + smaller models + batch
Hallucinations → Better context + strict prompting + reranking

Resources to Keep Bookmarked:

- LangChain Docs: python.langchain.com
- LlamaIndex Docs: gpt-index.readthedocs.io
- Vector DB Benchmarks: github.com/erik-dunteman/VectorDBBench
- Papers: arxiv.org (search "retrieval augmented generation")